
Building a Vietnamese DBpedia: Linked Open Data from Vietnamese Wikipedia

Ha Thi Kim Phung
20242460M

Do Ngoc Trung
20232311M

Bui Duc Phan Hoang
20252004M

Do Hoang Dung
20241184M

Abstract

DBpedia turns the semi-structured content of Wikipedia into Linked Data that can be queried with SPARQL and joined with other datasets. This report describes a DBpedia-style knowledge base for Vietnamese built from Vietnamese Wikipedia and Wikidata for a closed but densely connected domain: Vietnamese football (players, clubs, national teams and stadiums), provinces and universities. We (i) define an OWL ontology, `vio:`, with 18 classes and 54 properties, each class aligned to a DBpedia Ontology ancestor; (ii) select 990 entities through the Wikidata Query Service and collect their Vietnamese articles through the MediaWiki API; (iii) transform abstracts, infoboxes, categories and redirects into 88,898 RDF triples with stable IRIs that the project server dereferences, validated by eight quality checks and expanded by OWL 2 RL reasoning to 131,543 triples; (iv) link 777 entities to English DBpedia and all 990 to Wikidata with `owl:sameAs`, described as VoID linksets; and (v) serve the data through a SPARQL 1.1 Protocol endpoint, a command-line query client and a web application with a SPARQL editor, DBpedia-style entity pages, HTTP content negotiation and question answering in which a large language model writes the SPARQL query and each step is shown. Reasoning lets the data be queried with the DBpedia vocabulary (0 asserted `dbo:Person` instances become 659) and answers multi-hop questions such as “which clubs has a player played for” through a property chain. Checked against the live DBpedia endpoint, 733 of the 777 English links resolve to an article. The code, data and 152 automated tests are available at <https://github.com/PhungHaThiKim/vietnamese-dbpedia>.

Key words: DBpedia; Linked Open Data; Vietnamese Wikipedia; ontology; OWL 2 RL; SPARQL; Wikidata; knowledge graph question answering

1 Introduction

DBpedia extracts structured information from Wikipedia—infobox fields, abstracts, categories, redirects and links between articles—and publishes it as RDF, so that the content of an encyclopedia can be queried with SPARQL and joined with other datasets [1, 2]. The English edition sits at the centre of the Linked Open Data cloud, and language chapters publish the same kind of data for other Wikipedia editions [3]. Vietnamese Wikipedia articles carry the same semi-structured information: the infobox of a footballer lists his date of birth, birthplace and every club he played for, season by season, but as wikitext it cannot be queried. A question such as “*Which players born in Nghệ An have played for the national team, and for which club do they play now?*” requires reading and joining several articles by hand.

This project addresses Topic 2 of the course, *Build a DBpedia version for Vietnamese language*, which consists of five tasks: (1) define an ontology; (2) collect Vietnamese articles from Wikipedia;

Table 1: Where each task of the assignment is implemented. Paths are under `vidbpedia/`, except `ui/`.

Task	Implementation	Output	Section
1. Ontology	<code>ontology/*.ttl</code> , merged by <code>kg/ontology.py</code>	18 classes, 54 properties, 859 triples	3.2
2. Collect articles	<code>crawl/wikidata_seeds.py</code> , <code>crawl/wiki_enrich.py</code>	990 entities with their Vietnamese articles	3.3
3. 4-star data	<code>crawl/build_rdf.py</code> , <code>kg/postprocess.py</code>	131,543 triples in Turtle and N-Triples	3.4, 3.6
4. Link to DBpedia	enwiki sitelinks of the Wikidata item	777 <code>owl:sameAs</code> <code>dbp:</code> links, VoID linksets	3.5
5. Query interface	<code>web/</code> : endpoint, Linked Data, Gradio; <code>ui/</code> : React; <code>kg/query.py</code>	<code>/sparql</code> endpoint, <code>query</code> command, Linked Data routes, web interface	4.2, 4.1, 4.3

(3) transform the data into 4-star Linked Data; (4) establish links to the English DBpedia; and (5) provide an interface to query the data through a SPARQL endpoint or terminal.

Rather than extracting all of Vietnamese Wikipedia shallowly, we extracted a closed domain in depth: Vietnamese *football* (players, clubs, national teams, stadiums), *provinces* and *universities*. Its entities refer to one another, so the graph supports multi-hop questions: player → career station → club → home stadium → province, player → birth province, university → province. 777 of the 990 selected entities (78%) have an English Wikipedia article to link to, and 956 of the 990 articles (97%) contain an infobox, the main source that DBpedia extracts. Entities are selected through Wikidata rather than through the category tree of Vietnamese Wikipedia, whose categories editors assign inconsistently.

The contributions of this work are:

- an OWL ontology, `vio:`, with 18 classes and 54 properties, aligned to the DBpedia Ontology for OWL 2 RL reasoning (Section 3.2);
- a reproducible pipeline whose raw downloads are committed, so the dataset rebuilds offline (Sections 3.3–3.6);
- 131,543 triples (88,898 asserted, 41,730 inferred) that pass eight quality checks with no error or warning (Section 5);
- 777 `owl:sameAs` links to English DBpedia, verified on the live endpoint, and 990 to Wikidata (Sections 3.5 and 5.4);
- a server with a SPARQL 1.1 Protocol endpoint, IRIs dereferenced with 303 and content negotiation, a terminal client, and a web interface with entity pages, an ontology browser, a SPARQL editor and step-by-step question answering (Section 4).

Table 1 maps each task to its implementation; the following sections cover background, design, publishing, evaluation and discussion.

2 Background and Related Work

DBpedia. DBpedia [1, 2] extracts labels, abstracts, page identifiers, images, categories, redirects, external links, coordinates and infobox data from every Wikipedia article. Infobox data is published raw, as properties of the generic `dbp:` namespace, and mapping-based, through community mappings to the hand-built DBpedia Ontology (`dbo:`), which is cleaner but covers less. Resources are named after their article titles, and language chapters publish their own edition under their own namespace with links to the English one. Kontokostas et al. [3] describe what this raises for the Greek chapter: IRIs in a non-Latin script, language-specific templates and links across editions. We follow this model for Vietnamese, with a small ontology where DBpedia lacks the terms we need.

Linked Data. The Linked Data principles ask for HTTP URIs that return RDF when looked up and link to other URIs [4, 5]. The five-star scheme grades open data, and Tasks 3 and 4 correspond to its fourth and fifth stars. We serve the data with 303 *See Other* redirects and content negotiation [6, 7], describe it with VoID [8], and use RDF 1.1 [9] and SPARQL 1.1 [10].

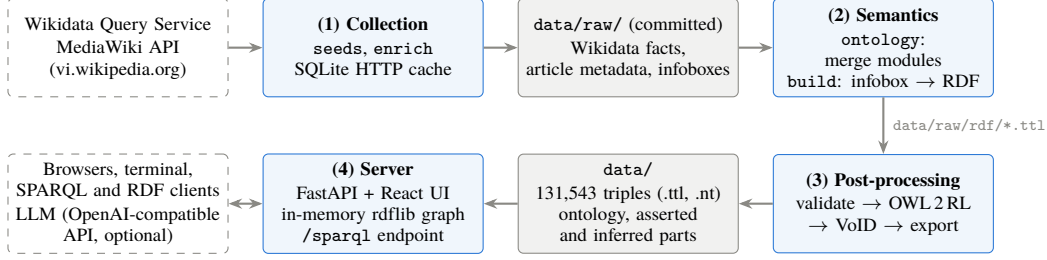


Figure 1: Architecture. Blue boxes are the four layers of the vidbpedia package (the interface of layer (4) is in ui/), gray boxes are files on disk, dashed boxes are external systems. Only layer (1) calls the network.

Table 2: Namespaces. The vi.dbpedia.org namespaces mirror the IRI scheme of DBpedia language chapters.

Prefix	Namespace	Used for
vio:	http://vi.dbpedia.org/ontology/	classes and properties of this project
vres:	http://vi.dbpedia.org/resource/	resources (entities)
vip:	http://vi.dbpedia.org/property/	raw infobox properties, like dbp:
vcat:	http://vi.dbpedia.org/resource/Thẻ_loại:	Wikipedia categories
dbo:	http://dbpedia.org/ontology/	DBpedia Ontology
dbr:	http://dbpedia.org/resource/	English DBpedia resources
wd:	http://www.wikidata.org/entity/	Wikidata items

Wikidata. Wikidata [11] items carry typed statements with qualifiers and *sitelinks* to the article about them in every Wikipedia edition, and the Wikidata Query Service exposes them through SPARQL [12]. We use it to select the entities, to obtain precise dates and quantities, and, through English sitelinks, to link to DBpedia.

Ontology engineering and question answering. The ontology was designed from *competency questions*, written before the schema and later used as tests [13]. DBpedia types and shortcut relations are derived with the OWL 2 RL profile [14], whose rules run by forward chaining in polynomial time, so the derived triples can be materialised. Retrieval-augmented generation grounds the answer of a language model in retrieved evidence [15]; over a knowledge graph the retrieval can be a SPARQL query written by the model, as on our question-answering page (Section 4.4).

3 System Design

3.1 Architecture

The system is a Python package, vidbpedia, organised in four layers (Figure 1). Each step is one command, `python -m vidbpedia <step>`: `seeds`, `enrich`, `ontology`, `build`, `postprocess` and `serve`, plus query for SPARQL from the terminal. Only the collection layer uses the network: HTTP responses are cached in SQLite and the raw downloads are committed under `data/raw/`, so the later layers rebuild the dataset offline and identically. Graphs are built with the rdflib API [16] rather than by concatenating Turtle strings, which rules out syntax and escaping errors, and a single function creates every resource IRI, so an entity cannot receive two IRIs.

3.2 Ontology

The ontology is written as ten Turtle modules in `ontology/`, from `000-prefixes.ttl` to `420-football.ttl`. They are merged in numeric order into `vi-ontology.ttl` (859 triples) and checked by rdflib when merged. Table 2 lists the namespaces. Following DBpedia [2], resources are named by their article title, infobox parameters have a raw namespace, and categories are SKOS concepts.

Classes. The ontology has two layers [5]. The upper layer reuses the DBpedia Ontology: `050-dbo-alignment.ttl` declares the 24 DBpedia classes and 44 properties that `vio:` maps

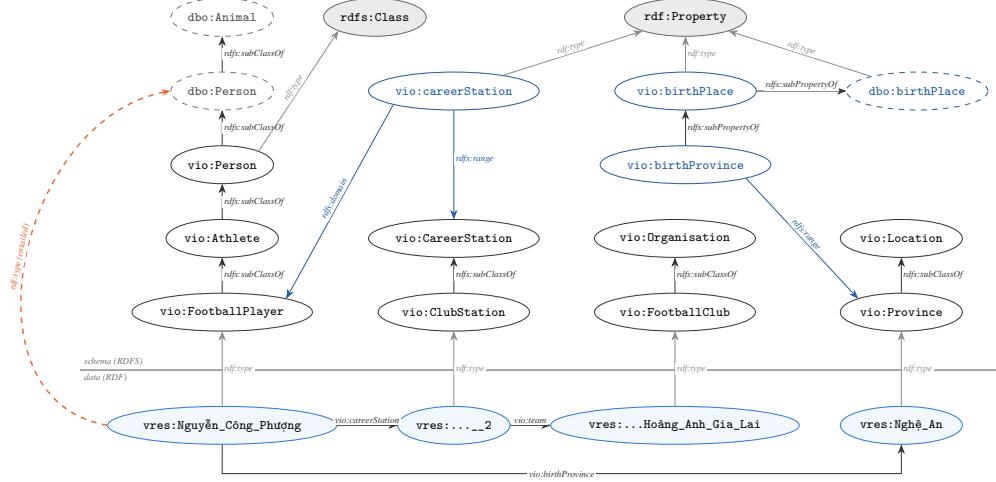


Figure 2: RDFS graph of an excerpt of the ontology. Above the line is the schema: classes and properties are resources typed by `rdfs:Class` and `rdf:Property`, linked by `rdfs:subClassOf` and `rdfs:subPropertyOf`; blue edges are the `rdfs:domain` and `rdfs:range` of two properties. Below the line are instance data (`vres:..._2` is the career station `vres:Nguyễn_Công_Phượng_2`). The dashed edge is a triple that RDFS entails: the player is also a `dbo:Person`. `dbo:Person` $\sqsubseteq$ `dbo:Animal` is DBpedia’s own axiom. Only some `rdf:type` edges to the RDFS vocabulary are drawn; the full class hierarchy is in Table 3.

Table 3: Class hierarchy of `vio:` (indentation shows `rdfs:subClassOf`), the DBpedia class each class is aligned to, and the number of instances before and after OWL 2 RL reasoning.

Class	DBpedia superclass	Asserted	After reasoning
Person	dbo:Person	0	659
Athlete	dbo:Athlete	0	611
FootballPlayer	dbo:SoccerPlayer	611	611
NationalTeamPlayer	—	0	486
Organisation	dbo:Organisation	0	410
FootballClub	dbo:SoccerClub	61	215
NationalFootballTeam	dbo:NationalSoccerClub	10	27
EducationalInstitution	dbo:EducationalInstitution	0	168
University	dbo:University	168	168
Location	dbo:Place	0	370
Province	dbo:Province	110	110
FormerProvince	—	76	76
Stadium	dbo:Stadium	29	44
Country	dbo:Country	1	1
CareerStation	dbo:CareerStation	0	3,382
YouthStation	—	527	527
ClubStation	—	1,966	1,966
NationalTeamStation	—	889	889

to, with DBpedia’s English labels, our Vietnamese labels (`dbo:Animal` is “Động vật”) and `rdfs:isDefinedBy` `dbo:`, but not DBpedia’s domains and ranges. The lower layer has 18 `vio:` classes in four disjoint trees: people, organisations, locations and career stations (Table 3). A `vio:` class exists only where an axiom or a competency question [13] needs it, such as `vio:FormerProvince`; there is no `vio:Animal`. Each class is an `rdfs:subClassOf`, not an `owl:equivalentClass`, of a DBpedia class, directly or through its parent. Data described with `vio:` can thus be queried with `dbo:` after reasoning (Section 3.6), while our domains, functional properties and disjointness axioms constrain only our own data, avoiding the Exclusivity anti-pattern [17]. `vio:FormerProvince` holds the provinces that were dissolved or merged, including in the 2025 reorganisation: a province is former if Wikidata gives a dissolution date (P576) or no current province statement (P31). With this rule, “how many provinces are there now” returns 34, while history is kept through `vio:successor` and `vio:predecessor`.

Properties. The ontology has 54 properties, 26 object and 28 datatype properties (Figure 2). All have labels in Vietnamese and English. Every datatype property has an `rdfs:range`, and 39 properties have an `rdfs:domain` (24 datatype, 15 object). When DBpedia has an equivalent of the same kind, the property is an `rdfs:subPropertyOf` of it (38 properties). Typical examples are `vio:birthDate` $\sqsubseteq$ `dbo:birthDate`, `vio:capacity` $\sqsubseteq$ `dbo:seatingCapacity`, and `vio:birthProvince` $\sqsubseteq$ `vio:birthPlace` $\sqsubseteq$ `dbo:birthPlace`. 22 datatype properties are functional, for example birth date, height and capacity. Some domains are left open on purpose. If `vio:latitude` or `vio:province` had the domain `Location`, every club and university that has coordinates or a province would be inferred to be a location, which contradicts the disjointness axioms.

Axioms. A career is modelled with the “role at a time” pattern of `dbo:CareerStation`. A player has one `vio:careerStation` per team and period, and the station carries the team, the start and end years, appearances, goals and an on-loan flag. Four kinds of axiom act on this structure during reasoning:

- a *property chain*, `vio:careerStation` $\circ$ `vio:team` $\sqsubseteq$ `vio:playedFor`, which gives a direct player–team edge;
- *inverses*, so that each relation can be queried in both directions: `vio:hasPlayer` is the inverse of `vio:playedFor`, and the same holds for the pairs `vio:hasPart`/`vio:isPartOf` and `vio:predecessor`/`vio:successor`;
- *restrictions*, which derive classes from the career data:

`FootballPlayer` $\sqcap \exists$ `careerStation.NationalTeamStation` $\sqsubseteq$ `NationalTeamPlayer`,
`ClubStation` $\sqsubseteq$ $\forall$ `team.FootballClub`.

The intersection keeps the first definition independent of the domain of `careerStation` [17]. Similar $\forall$ restrictions hold for youth and national-team stations. They give a class to teams that are not seeds, such as the Japanese club Mito HollyHock;

- four *disjointness* axioms (`owl:AllDisjointClasses`): between the four roots, between club, national team and educational institution, between province, stadium and country, and between the three kinds of station. They let the reasoner detect modelling errors (Section 3.6).

3.3 Data Collection

Entity selection. `seeds` runs one Wikidata Query Service query per class. A query matches the class through `P31/P279*` and the country through `P17` or `P27` (Vietnam, `Q881`). It keeps only items with a Vietnamese Wikipedia article, and makes the English article optional. The query for players is shown below. Wikimedia list pages, disambiguation pages and titles starting with “Danh sách” (list of) are removed. When an item matches several classes, the first class in the priority order wins.

```
SELECT DISTINCT ?item ?viTitle ?enTitle WHERE {
  ?item wdt:P31 wd:Q5 ; wdt:P106 wd:Q937857 ; wdt:P27 wd:Q881 . # footballer, Vietnam
  FILTER NOT EXISTS { ?item wdt:P31 wd:Q13406463 } # Wikimedia list page
  FILTER NOT EXISTS { ?item wdt:P31 wd:Q4167410 } # disambiguation page
  ?vs schema:about ?item ; schema:isPartOf <https://vi.wikipedia.org/> ;
    schema:name ?viTitle .
  OPTIONAL { ?es schema:about ?item ; schema:isPartOf <https://en.wikipedia.org/> ;
    schema:name ?enTitle . } }
```

Facts are fetched in blocks of at most 300 items: dates with their precision, so that a year-only birth date becomes `xsd:gYear` instead of a fabricated 1 January; quantities (height, capacity, population, area, students) with their point in time (`P585`), so that the latest value is kept; relations with their start, end, appearance and goal qualifiers; and the labels and sitelinks of every referenced item. Birthplaces and locations are mapped to a province through `P131*`. The result is 990 seeds: 611 players, 61 clubs, 10 national teams, 29 stadiums, 168 universities, 110 provinces and 1 country.

Article collection. `enrich` reads the Vietnamese articles through the MediaWiki Action API, in batches of up to 50 titles with `maxlag` set. It stores the page ID, revision, length, coordinates, lead image, plain-text introduction (the abstract), non-hidden categories and incoming redirects, and, instead

Table 4: DBpedia datasets reproduced for every entity, with their size in the current build.

Dataset	Properties	Size
Labels	<code>rdfs:label @vi, @en</code>	990 vi, 805 en
Abstracts	<code>dbo:abstract</code> (lead section), <code>rdfs:comment</code> (at most 400 characters)	990
Page metadata	<code>dbo:wikiPageID</code> , <code>dbo:wikiPageRevisionID</code> , <code>prov:wasDerivedFrom</code> , <code>foaf:isPrimaryTopicOf</code>	990
Images	<code>dbo:thumbnail</code> , <code>foaf:depiction</code>	543
Categories	<code>dct:subject</code> → <code>skos:Concept</code>	9,126 links, 1,154 categories
Redirects	<code>dbo:wikiPageRedirects</code> , <code>skos:altLabel</code>	1,274
External links	<code>dbo:wikiPageExternalLink</code> , <code>foaf:homepage</code>	713 links, 270 homepages
Coordinates	<code>vio:latitude/vio:longitude</code> ($\sqsubseteq$ <code>wgs84:</code>), <code>georss:point</code>	189 entities
Interlinks	<code>owl:sameAs dbr:</code> , <code>wd:</code>	777, 990
Raw infobox	<code>vip:key</code>	24,605 triples, 687 properties

of the whole wikitext, the parameters of the outermost infobox (found with `mwparserfromhell`) and the URLs of the “Liên kết ngoài” (external links) section. Every title linked from an infobox is resolved, following redirects and flagging disambiguation pages (2,588 titles, 71 disambiguation pages), and redirects to the infobox templates are kept as template aliases. Requests are spaced 2 s apart for the Wikidata Query Service and 1 s for Wikipedia, carry a contact User-Agent, and are retried on HTTP 429 and `maxlag`; `--offline` and `--refresh` use only the cache or bypass it.

3.4 Transformation to RDF

IRIs. A resource IRI is `vres:` followed by the Vietnamese title as MediaWiki normalises it: NFC Unicode, first character in upper case, spaces as underscores. Like DBpedia chapters, the IRI keeps Unicode characters, for example `vres:Nguyễn_Công_Phước`, and percent-encodes only the characters that are not allowed in an IRI. Career stations are numbered from youth to club to national team (`vres:Nguyễn_Công_Phước__3`). Redirects are separate resources that point to the main resource with `dbo:wikiPageRedirects`, and their titles are also added to the main resource as `skos:altLabel`, so that it can be found by its other names.

DBpedia datasets. For every entity, build produces the same basic datasets as DBpedia (Table 4). Raw infobox values are kept under `vip:`, as in DBpedia’s `dbp:` namespace.

Mapping-based infobox extraction. Infobox values are mapped to `vio:` properties in four stages. The wikitext of each value is first *rendered*: common nested templates (dates, heights, URLs and the list templates `ubl`, `plainlist` and `hlist`) are interpreted, and references, comments and flag icons removed. It is then *parsed* into a typed literal: the parsers understand Vietnamese numbers (16.361,2), open year ranges (2015–) and loan markers (→, (mượn)), and accept heights only between 1.4 and 2.2 m. Keys are *mapped* to `vio:` properties by the MAPPINGS table of five template families (players, clubs and national teams, stadiums, universities, provinces); as in the mappings wiki of DBpedia, a property can take keys from both the Vietnamese and the English template. Finally the PRECEDENCE table *chooses a source* when Wikidata and the infobox both have a value: Wikidata wins for dates, height, population, area, capacity, opening year, homepage and abbreviation, the infobox for shirt number, numbers of students and academic staff, motto and administrative code, and object values of both are combined. Link targets are filtered by class: the team of a career station cannot be a province, a stadium or a university, and the `vio:ground` of a club must be a stadium.

Career stations come from the numbered parameters of the football biography infobox (`yearsn`, `clubsn`, `capsn` and `goalsn`) and their youth and national variants. For 594 of the 611 players the career is read from the infobox; the other 17 have no club rows (or no infobox) and fall back to the Wikidata statements P54. In total 601 players receive 3,382 stations. The build also writes `mapping_stats.json`, which counts mapped and unmapped keys for each template family, to guide which mappings to add next.

```

{{Thông tin tiểu sử bóng đá
| birth_date   = {{ngày sinh và tuổi|1995|1|21}}<ref name=a/>
| birth_place = [[Đô Lương (huyện)|Đô Lương]], [[Nghệ An]], [[Việt Nam]]
| height      = {{convert|1,69|m}}
| currentclub = [[Câu lạc bộ bóng đá Thành phố Đồng Nai|Thành phố Đồng Nai]]
| clubnumber  = 10
| years2     = 2016    | clubs2 = → [[Mito HollyHock]] (mượn)    | caps2 = 5    | goals2 = 0

```

↓ python -m vidbpedia build

```

vres:Nguyễn_Công_Phượng a vio:FootballPlayer ;
  vio:birthDate "1995-01-21"^^xsd:date ;
  vio:birthPlace vres:Đô_Lương_(huyện) ;
  vio:birthProvince vres:Nghệ_An ; # Wikidata P19, then P131*
  vio:height 1.68e+00 ; # xsd:double, from Wikidata
  vio:currentClub vres:Câu_lạc_bộ_bóng_đá_Thành_phố_Đồng_Nai ;
  vio:shirtNumber "10"^^xsd:nonNegativeInteger ;
  vio:careerStation vres:Nguyễn_Công_Phượng__3 .
vres:Nguyễn_Công_Phượng__3 a vio:ClubStation ;
  vio:team vres:Mito_HollyHock ; vio:onLoan true ;
  vio:startYear "2016"^^xsd:gYear ; vio:endYear "2016"^^xsd:gYear ;
  vio:appearances "5"^^xsd:nonNegativeInteger ; vio:goals "0"^^xsd:nonNegativeInteger .

```

Listing 1: From three to four stars: part of the infobox of Nguyễn Công Phượng (top, wikitext) and the triples the build produces for it (bottom, Turtle, excerpt).

From three to four stars. Listing 1 follows one infobox through the transformation. Before it, the data are machine-readable and open but name nothing with a URI: at most three stars. After it, they are four-star RDF, through four changes. *Every thing gets an HTTP IRI*: the player, the district, the province, both clubs and even the loan spell, `vres:Nguyễn_Công_Phượng__3`. *Values become typed literals* (`xsd:date`, `xsd:gYear`, `xsd:nonNegativeInteger`, `xsd:double`, `xsd:boolean`) and texts carry a language tag. *Markup is interpreted*: templates such as `ngày sinh và tuổi` (date of birth and age) become values, links are resolved through redirects, the loan marker `→ ... (mượn)` becomes `vio:onLoan true`, and PRECEDENCE takes the height of 1.68 m from Wikidata over the infobox's 1.69 m. *Terms come from shared vocabularies*: `vio:` terms specialise `dbo:` terms, and the core datasets use `rdfs:`, `skos:`, `foaf:`, `dct:` and `prov:`.

3.5 Interlinking

The English link of an entity comes from its Wikidata item: if the item has an English sitelink, the resource receives `owl:sameAs` to the `dbr:` resource of that title, written in the same Unicode IRI form that DBpedia uses. Using the sitelink of the same item avoids matching by label, which would be ambiguous for common Vietnamese names. Every entity also receives `owl:sameAs` to its Wikidata item. In addition to these instance links, the ontology is linked to DBpedia at the schema level through its subclass and sub-property axioms.

The dataset describes itself with VOID [8]: a `void:Dataset` with the licence (CC BY-SA 4.0, as Wikipedia), sources, URI space, vocabularies and dump location; three subsets by provenance (ontology, asserted, inferred) with their triple counts; a class partition for each of the seven entity classes; and two `void:Linksets`, to DBpedia and to Wikidata, with the number of links.

3.6 Validation, Reasoning and Export

Validation. `postprocess` runs eight checks before reasoning (Table 5). Each reports issues with severity, subject and detail; the tests run the same functions, and with `--strict` any error stops the build.

Reasoning. Reasoning uses the OWL 2 RL rules of the `owlrl` library [14, 16] and runs in 35 s. Three choices keep it fast and correct. Functional-property, cardinality and `owl:sameAs` axioms are removed from the TBox, because OWL 2 RL would otherwise derive `x owl:sameAs x` for every node and merge entities that share a functional value; functional properties are checked by validation

Table 5: Quality checks (kg/validation.py). The last build reports no error and no warning.

Check	Condition
tbox	sub-properties link properties of the same kind; every vio: class has a dbo: ancestor; terms used in axioms are declared; declared terms have vi and en labels and rdfs:isDefinedBy; vio: names are CamelCase
property_kinds	object properties point to IRIs, datatype properties to literals
datatype_ranges	literals match the declared rdfs:range (including rdf:langString) and are well-formed
required	every entity has exactly one main class, a Vietnamese label, a source page and a Wikidata link
functional	functional properties have at most one value
object_ranges	if the target of an object property has a class, that class fits the range
sameas	no list or disambiguation targets; no two resources share a target
plausibility	coordinates inside Vietnam; birth year, capacity, population and height in a plausible range (warnings)

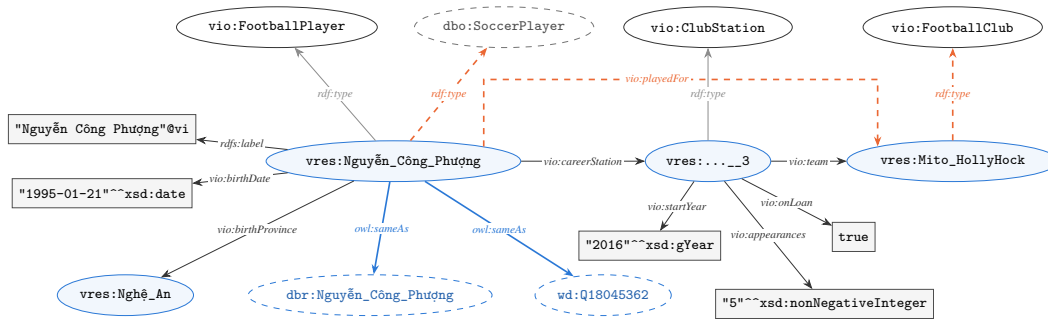


Figure 3: Nguyễn Công Phượng and one of his eleven career stations as an RDF graph, drawn as in the lectures: ellipses are resources (entities and classes), rectangles are literals, and each edge is one triple labelled with its property. Gray and black edges are asserted. Orange dashed edges are inferred by OWL 2 RL: `vio:playedFor` through the property chain, `dbo:SoccerPlayer` through the subclass axioms, and the class of Mito HollyHock through the $\forall$ restriction of `vio:ClubStation`. Blue edges are `owl:sameAs` links to other datasets. `vres:..._3` is `vres:Nguyễn_Công_Phượng__3`, the loan spell of Listing 1.

instead. Only the *skeleton* of the data reaches the reasoner, the `rdf:type` triples and the `vio:` triples of `vres:` resources, since abstracts, images and raw infobox triples cannot trigger any rule. Only new triples whose class or property is in `vio:`, `dbo:` or `wgs84:` are kept, never `owl:sameAs`, and they are written to their own file, `inferred.nt`, so the interface can label each triple as asserted or inferred. Reasoning also helped us find two modelling errors. First, it reported a violation of the disjointness between clubs and national teams. The cause was that `vio:ground`, `vio:manager` and `vio:chairman` had the domain `FootballClub`, so every national team with a home ground was inferred to be a club; the domain is now `Organisation`. Second, the question “which clubs have their home ground in Hà Nội” returned clubs from Đà Nẵng and Khánh Hòa. The cause was an axiom declaring `vio:tenant` the inverse of `vio:ground`, while the “tenants” field of the Hàng Đẫy stadium lists every team that ever played there. We removed the axiom, as DBpedia also keeps `dbo:tenant` and `dbo:ground` separate, and kept only tenants that are entities of the dataset. Figure 3 shows asserted and inferred triples around one player.

Export. The full dataset (ontology, asserted and inferred triples, VoID) is written as Turtle and N-Triples, and as RDF/XML with `--rdfxml`; the three parts are also written separately. The N-Triples file is read back and its triple count compared with the graph in memory, and statistics and the validation report are written as JSON.

4 Publishing and Querying

`python -m ui.api` (port 8000) starts one FastAPI application that hosts the SPARQL endpoint, the Linked Data routes and a React interface (`ui/`), all on the same graph; the earlier Gradio [18] interface

Table 6: Linked Data routes (`web/linked_data.py`). An unknown name returns 404.

Route	Response
GET <code>/resource/{name}</code>	303 to <code>/?resource={name}</code> for browsers, or to <code>/data/{name}.ttl</code> , <code>.nt</code> , <code>.jsonld</code> or <code>.rdf</code> when <code>Accept</code> asks for Turtle, N-Triples, JSON-LD or RDF/XML; sends <code>Vary: Accept</code>
GET <code>/page/{name}</code>	303 to the HTML page, like <code>dbpedia.org/page/...</code>
GET <code>/data/{name}.{ext}</code>	every triple with the resource as subject, and up to 2,000 with it as object, in the requested format; sends <code>Access-Control-Allow-Origin: *</code>
GET <code>/ontology/{term}</code>	the definition of a <code>vio:</code> class or property, in Turtle

```

$ curl -si -H "Accept: text/turtle" \
  http://127.0.0.1:8000/resource/Nguy%E1%BB%85n_C%C3%B4ng_Ph%C6%B0%E1%BB%A3ng
HTTP/1.1 303 See Other
vary: Accept
location: /data/Nguy%E1%BB%85n_C%C3%B4ng_Ph%C6%B0%E1%BB%A3ng.ttl

$ curl -sL -H "Accept: text/turtle" http://127.0.0.1:8000/resource/Nguy%E1%BB%85n_...
vres:Mito_HollyHock vio:hasPlayer vres:Nguyễn_Công_Phượng .
...
vres:Nguyễn_Công_Phượng a dbo:Animal, dbo:Athlete, dbo:Person, dbo:SoccerPlayer, ... ;
  rdfs:label "Nguyễn Công Phượng"@en, "Nguyễn Công Phượng"@vi ;
  ...
  vio:birthProvince vres:Nghệ_An ;
  vio:careerStation vres:Nguyễn_Công_Phượng__1, ... ;
  ...
  owl:sameAs dbr:Nguyễn_Công_Phượng, wd:Q18045362 ;

```

Listing 2: Dereferencing a resource IRI with `curl`. Response headers are abridged and the Turtle is an excerpt.

is still served by `python -m vidbpedia serve`. At start-up the server loads the full N-Triples dump into an in-memory `rdflib` graph, loads `inferred.nt` into a set so that every triple can be labelled as asserted or inferred, builds a search index over labels and alternative names that ignores Vietnamese diacritics, precomputes the class tree, and loads a second graph without the inferred triples.

4.1 Dereferenceable IRIs

The server follows the pattern of DBpedia and of the W3C Cool URIs note [6]. Looking up a resource IRI never returns content directly. It answers *303 See Other* and redirects either to an HTML page or to an RDF document, depending on the `Accept` header (Table 6). The HTML page is `/?resource=<name>`, which opens the entity page `/entity/<name>`, so pages can be shared.

Listing 2 shows an exchange recorded on the running server. As browsers do, clients must percent-encode the non-ASCII characters of the path. In the response, the `dbo:*` types and the edge `vio:hasPlayer` are inferred; `dbo:Animal` appears because the DBpedia Ontology places `dbo:Person` under `dbo:Animal`, and our alignment module copies that class chain.

4.2 SPARQL Endpoint and Terminal Client

The route `/sparql` implements the SPARQL 1.1 Protocol [19], so that `curl`, `SPARQLWrapper` or `YASGUI` can query the graph. It takes the query as a GET parameter, a POST form field or an `application/sparql-query` body, with the 19 project prefixes predeclared, and returns SPARQL JSON (the default), XML or CSV for `SELECT` and `ASK`, and Turtle, N-Triples, JSON-LD or RDF/XML for `CONSTRUCT` and `DESCRIBE`, chosen by a format parameter, as on DBpedia, or by the `Accept` header. It is read-only, answers an invalid query with 400 and the error, rejects `FROM` and `SERVICE`, which would fetch other URLs, and runs queries in a worker thread. The non-standard parameter `inference=false` restricts a query to asserted triples (0 `dbo:Person` instances instead of 659).

The command `python -m vidbpedia query` runs the same queries from a terminal, on the loaded dump or, with `--endpoint`, through a running endpoint (Listing 3). Both use one mod-

```
$ curl -H "Accept: text/csv" http://127.0.0.1:8000/sparql --data-urlencode "query=
  SELECT ?stadium ?seats WHERE { ?stadium a dbo:Stadium ; dbo:seatingCapacity ?seats }
  ORDER BY DESC(?seats) LIMIT 3"
stadium,seats
http://vi.dbpedia.org/resource/Sân_vận_động_Cần_Thơ,44400
http://vi.dbpedia.org/resource/Sân_vận_động_Quốc_gia_Mỹ_Đinh,40192
http://vi.dbpedia.org/resource/Sân_vận_động_Thiên_Trường,30000
$ python -m vidbpedia query "SELECT ?province (COUNT(?p) AS ?players) WHERE {
  ?p vfo:birthProvince ?province } GROUP BY ?province ORDER BY DESC(?players) LIMIT 3"
province      | players
-----+-----
vres:Nghệ_An  | 58
vres:Hà_Nội   | 48
vres:Thanh_Hóa | 42
(3 dòng)
```

Listing 3: The SPARQL endpoint called with `curl` (CSV results) and the terminal client (table). The first query uses only DBpedia terms, so every answer comes from inferred triples.

The screenshot shows the entity page for Nguyễn Công Phượng. At the top, there's a profile picture and the name 'Nguyễn Công Phượng'. Below it, a description says 'Thực thể thuộc lớp Cầu thủ bóng đá' and provides a URI. There are links to Wikipedia, DBpedia EN, Wikidata, and Linked Data (Turtle). A note indicates '122 triple khai báo' and '38 triple suy luận'. On the left, a 'Cây phân lớp' (Class Hierarchy) shows the path from 'dbo:Animal' to 'vfo:NationalTeamPlayer'. On the right, a 'Liên kết dữ liệu mở' (Open Data Links) section lists various RDF links and a 'URI dereference' button.

Figure 4: Entity page of Nguyễn Công Phượng (abstract hidden): the asserted class and the named graph, classes marked asserted (*khai báo*) or inferred (*suy luận*), and the Linked Data card. Labels are in Vietnamese.

ule, `kg/query.py`, which replaces two serializers of `rdflib 7.6`: its XML results writer drops the literals `0` and `false`, and its text table loses the `ORDER BY`.

4.3 Web Interface

The interface has five pages (Figures 4–6). A top-bar switch hides inferred triples in every graph.

- **Overview** summarises the dataset, its inferred share, the links and the validation result.
- **Ontology** lists all 8,624 resources under their most specific `vfo:` class, DBpedia superclasses after a $\sqsubseteq$ (a root shows its chain, `vfo:Person` $\sqsubseteq$ `dbo:Person` $\sqsubseteq$ `dbo:Animal`), and the OWL axioms with the triples each produced.
- **Entity** renders a page modelled on `dbpedia.org/page/...`: links to Wikipedia, DBpedia and Wikidata, the class tree with an asserted or inferred badge for each class, the `owl:sameAs` and provenance links, RDF downloads in four formats, the key facts, a career timeline, a draggable neighbourhood graph, a relation tree up to three hops deep and the property tables.
- **Question answering** (Section 4.4).
- **SPARQL** is an editor with the 19 prefixes predeclared, nine example queries, a switch for inferred triples and JSON or CSV downloads.

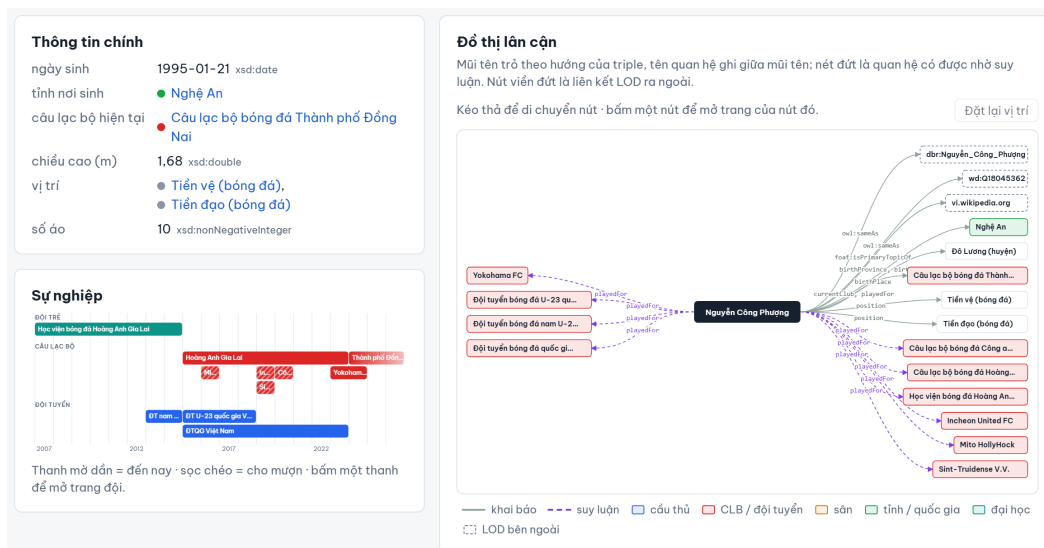


Figure 5: Lower part of the same page: key facts, career timeline and the draggable neighbourhood graph, where the playedFor edges are dashed because they are inferred.

4.4 Natural-Language Question Answering

A language model writes a SPARQL query from a prompt with a schema summary, rules (never guess IRIs, use the playedFor shortcut, skip former provinces) and six example queries. A query that fails or returns no rows goes back once with the error, and a second prompt answers from at most 30 rows; any OpenAI-compatible API works.

Entity linking. Step 1 needs no model. The labels (Vietnamese and English) and redirect titles of the 1,176 entities of the main classes are folded to lower case without diacritics and stored in two hash tables: whole names, also without a bracketed qualifier, and the last two to four words of each name, since Vietnamese names start with the type or the family name (“Nguyễn Quang Hải”). Spans of the question are looked up longest first; a matched word is not reused, spans of generic words are skipped, and a name shared by up to six entities keeps them all. Linking takes about 1 ms.

The page shows the answer above the six steps behind it: (1) entity linking, whose IRIs are added to the prompt; (2) each generation attempt, with feedback naming IRIs absent from the graph or of the wrong class; (3) checks of syntax, read-only form, FROM, SERVICE, LIMIT and linked IRIs, with the inferred share of each ontology term; (4) execution with and without inferred triples; (5) the answer; and (6) an evidence graph (Figure 6). The examples follow one performance rule: rdflib applies a FILTER only after joining the whole group, so the lookup of an entity by name is written as a subquery that runs first. For the career of Nguyễn Công Phượng this takes the query from 10.7 s to 0.27 s.

5 Evaluation

All figures in this section are computed from the published dataset by report/report_numbers.py.

5.1 Dataset and Coverage

The dataset contains 131,543 triples: 88,898 asserted, 41,730 inferred, 859 in the ontology and 56 in the VoID description. They describe 990 entities, 3,382 career stations (527 youth, 1,966 club and 889 national-team stations), 1,154 categories and 1,274 redirects, for 8,624 distinct resource IRIs in total. 1,382 of these resources have a label but no class. They are targets of links in infobox values, such as cities, districts, people and competitions. Table 7 shows the coverage by class.

Infobox coverage is high for every class except national teams, whose Vietnamese articles often lack an infobox. Links to English DBpedia are complete for clubs and national teams but cover about

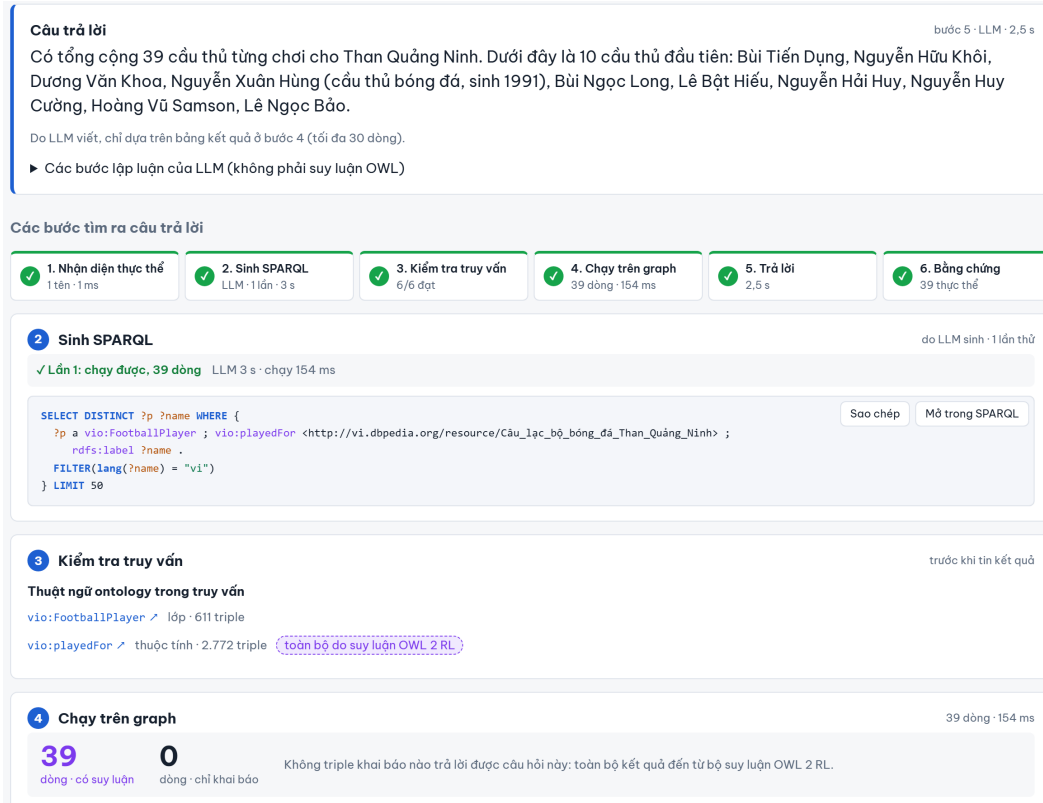


Figure 6: Question answering with gpt-4o for “which players have played for Than Quảng Ninh?”: the answer, the progress of the six steps, the generated query, the ontology terms with their inferred share, and 39 rows with reasoning against 0 without (step 1 and the checks of step 3 hidden). Labels are in Vietnamese.

Table 7: Entities by class, articles with an infobox, and entities linked to English DBpedia.

Class	Entities	With infobox	Linked to DBpedia
Football players	611	602	508
Football clubs	61	59	60
National teams	10	5	10
Stadiums	29	29	25
Universities	168	159	89
Provinces (34 current)	110	101	84
Country	1	1	1
Total	990	956 (97%)	777 (78%)

half of the universities, many of which have no English article. Player properties are well covered: position for 586 of 611 players, birth date for 572, height for 534, birth province for 462 and current club for 442; 56 of 61 clubs have a home ground and 142 of 168 universities a province. 129 of the 3,382 career stations (3.8%) have no `vio:team`, because the team is plain text in the infobox or its link was rejected by the class filter.

5.2 Effect of Reasoning

Table 8 compares queries on the asserted data alone with the same queries on the published dataset. Of the 41,730 inferred triples, 18,341 are types and 23,389 are property values. Without reasoning, none of the DBpedia classes or properties has an instance: the data uses only the `vio:` vocabulary. After reasoning, a client that knows only DBpedia can ask for `dbo:SoccerPlayer` or `dbo:team` and get answers. The 48 people who are not players (coaches, chairmen and rectors) receive `dbo:Person` from the range of the property that links to them. The property chain turns a two-step path through

Table 8: Answers to the same query on the asserted data alone and on the published dataset (asserted plus inferred).

Query pattern	Asserted	Published	Inferred through
?x a dbo:Person	0	659	subclass, rdfs:range
?x a dbo:SoccerPlayer	0	611	vio:FootballPlayer $\sqsubseteq$ dbo:SoccerPlayer
?x a dbo:Organisation	0	410	subclass
?x a dbo:Place	0	370	subclass, rdfs:range
?x vio:playedFor ?t	0	2,772	chain careerStation o team
?t vio:hasPlayer ?x	0	2,772	owl:inverseOf
?x dbo:team ?t	0	3,699	sub-property
?x dbo:birthPlace ?p	0	802	sub-property

Table 9: owl:sameAs targets in English DBpedia, checked against dbpedia.org/sparql on 7 October 2026.

Status of the dbr: target	Links	Share
Article	733	94.3%
Redirect (dbo:wikiPageRedirects)	12	1.5%
Not in DBpedia	32	4.1%
Total	777	100%

career stations into 2,772 direct vio:playedFor edges. The class counts of Table 3 show the effect of the restrictions: 486 players become vio:NationalTeamPlayer, and 154 clubs that are not seeds, such as Mito HollyHock, become vio:FootballClub because they are the team of a club station.

5.3 Quality and Tests

The eight checks of Table 5 report no error and no warning on the last build. The test suite has 62 tests and passes in about 25 s: 12 competency questions on the real data (Section 5.5); 5 quality tests, including the ontology conventions; 4 reasoning tests on small graphs; 6 parser tests; 5 question-answering tests with a stubbed model; 19 tests of the Linked Data routes, resource page and resource tree; and 11 tests of the endpoint and terminal client, including the rejection of updates, FROM and SERVICE. The interface adds 90 tests on the same data. Tests use lower bounds instead of exact counts, so they keep passing after a new crawl.

5.4 Links to English DBpedia

We sent every dbr: target to the public DBpedia SPARQL endpoint and checked whether it describes an article, a redirect or nothing (Table 9). 733 of the 777 links (94.3%) point to an article, which also confirms that DBpedia uses the same Unicode IRIs as we do. Twelve point to a redirect (for example dbr:Quảng_Ninh) and 32 to titles that DBpedia does not contain, half of them clubs with recent titles such as dbr:Ho_Chi_Minh_City_FC_(2025): our links come from today’s English Wikipedia, while DBpedia was extracted from an older snapshot.

5.5 Competency Questions

Table 10 lists competency questions that guided the ontology, with the answers that the graph returns. Each question is a test in tests/test_sparql.py, and most need several hops or an inferred triple.

5.6 Five-Star Assessment

Table 11 assesses the dataset against the five-star scheme [4].

6 Discussion

Design decisions. Besides the selection through Wikidata and the two-layer ontology (Sections 3.3 and 3.2), inferred triples are *materialised and stored separately*, so clients get DBpedia types without

Table 10: Competency questions and the answers returned by the graph.

Question	Answer	Relies on
How many provinces and centrally governed cities are there now?	34 (of 110 provinces, 76 are former)	<code>vio:FormerProvince</code>
Which unit succeeded the former province of Hà Tây?	Hà Nội	<code>vio:successor</code>
Which teams has Nguyễn Công Phượng played for?	11 teams over 11 stations, 7 of them at clubs	property chain
Which provinces are home to the most players?	Nghệ An 58, Hà Nội 48, Thanh Hóa 42	<code>vio:birthProvince</code>
How many universities are in Hà Nội?	54	<code>vio:province</code>
How many clubs have a home ground in a known province?	44	<code>club</code> → <code>stadium</code> → <code>province</code>
Which entity is meant by “Công Phượng”?	Nguyễn Công Phượng	<code>dbo:wikiPageRedirects</code>
Which entities are DBpedia soccer players?	611	subclass reasoning

Table 11: The dataset against the five-star scheme.

Star	Evidence
★ open licence on the web	CC BY-SA 4.0, declared in the VoID description and the ontology header; dumps published in a Git repository
★★ machine-readable	RDF triples, not text or tables
★★★ open formats	Turtle and N-Triples dumps; each resource also as JSON-LD and RDF/XML; query results in the SPARQL JSON, XML and CSV formats
★★★★ URIs for things	an HTTP IRI for every entity, career station, category and redirect; dereferenced with 303 and content negotiation by the project server (Section 4.1)
★★★★★ linked	777 <code>owl:sameAs</code> links to English DBpedia, 990 to Wikidata, described as VoID linksets; classes and properties aligned to <code>dbo:</code>

a reasoner and the interface can mark what is inferred; and *one Python process with an in-memory rdflib graph* serves the endpoint, the Linked Data routes and the interface. The 131 thousand triples load in a few seconds; a triple store such as Virtuoso pays off when the data grows.

Limitations. The current system has six limitations.

- *Coverage.* 990 entities of one domain, not all of Vietnamese Wikipedia; each new class needs a seed query, ontology terms, an infobox mapping and a build function.
- *Resolvable IRIs.* We do not control `vi.dbpedia.org`, so the IRIs dereference only through a running project server; a domain of our own or `w3id.org` would make them resolvable on the Web.
- *Link drift.* 44 of the 777 English links do not reach a DBpedia article (Section 5.4); following `dbo:wikiPageRedirects` would fix twelve of them.
- *Data gaps.* 129 career stations have no team IRI, 26 universities have no province, and page links (`dbo:wikiPageWikiLink`) and `skos:broader` are not extracted.
- *Serving.* The SPARQL endpoint has no query time limit, because `rdflib` cannot cancel a running query; a public deployment needs a stoppable process or a triple store that enforces a limit.
- *Question answering.* Its quality depends on the language model and was not measured on a benchmark; the evidence graph is empty when the rows hold one end of each relation.

7 Conclusion

We built a DBpedia-style knowledge base for Vietnamese football, provinces and universities that covers the five tasks of the assignment: a two-layer OWL ontology aligned to the DBpedia Ontology;

990 entities selected through Wikidata; 88,898 validated triples, which OWL 2 RL reasoning extends to 131,543; 777 links to English DBpedia (733 resolve) and 990 to Wikidata; and a SPARQL endpoint, a terminal client, dereferenceable IRIs and a web interface with question answering. Reasoning makes the data interoperable: it adds the DBpedia types that clients expect and shortcut edges for multi-hop questions.

Future work follows from the limitations: query time limits or a triple store, more classes (coaches, competitions) and page links, a link check against DBpedia at every build, and a Vietnamese question-answering benchmark.

References

- [1] S. Auer, C. Bizer, G. Kobilarov, J. Lehmann, R. Cyganiak, and Z. Ives, “DBpedia: A nucleus for a web of open data,” in *Proc. 6th International Semantic Web Conference (ISWC 2007)*, ser. LNCS, vol. 4825. Springer, 2007, pp. 722–735.
- [2] J. Lehmann, R. Isele, M. Jakob, A. Jentzsch, D. Kontokostas, P. N. Mendes, S. Hellmann, M. Morsey, P. van Kleef, S. Auer, and C. Bizer, “DBpedia – a large-scale, multilingual knowledge base extracted from Wikipedia,” *Semantic Web*, vol. 6, no. 2, pp. 167–195, 2015.
- [3] D. Kontokostas, C. Bratsas, S. Auer, S. Hellmann, I. Antoniou, and G. Metakides, “Internationalization of linked data: The case of the Greek DBpedia edition,” *Journal of Web Semantics*, vol. 15, pp. 51–61, 2012.
- [4] T. Berners-Lee, “Linked data – design issues,” W3C Design Issues, 2006, the five-star scheme was added in 2010. [Online]. Available: <https://www.w3.org/DesignIssues/LinkedData.html>
- [5] T. Heath and C. Bizer, *Linked Data: Evolving the Web into a Global Data Space*, ser. Synthesis Lectures on the Semantic Web: Theory and Technology. Morgan & Claypool, 2011.
- [6] L. Sauermann and R. Cyganiak, “Cool URIs for the semantic web,” W3C Interest Group Note, 2008. [Online]. Available: <https://www.w3.org/TR/cooluris/>
- [7] B. Hyland, G. Atezing, and B. Villazón-Terrazas, “Best practices for publishing linked data,” W3C Working Group Note, 2014. [Online]. Available: <https://www.w3.org/TR/ld-bp/>
- [8] K. Alexander, R. Cyganiak, M. Hausenblas, and J. Zhao, “Describing linked datasets with the VoID vocabulary,” W3C Interest Group Note, 2011. [Online]. Available: <https://www.w3.org/TR/void/>
- [9] R. Cyganiak, D. Wood, and M. Lanthaler, “RDF 1.1 concepts and abstract syntax,” W3C Recommendation, 2014. [Online]. Available: <https://www.w3.org/TR/rdf11-concepts/>
- [10] S. Harris and A. Seaborne, “SPARQL 1.1 query language,” W3C Recommendation, 2013. [Online]. Available: <https://www.w3.org/TR/sparql11-query/>
- [11] D. Vrandečić and M. Krötzsch, “Wikidata: a free collaborative knowledgebase,” *Communications of the ACM*, vol. 57, no. 10, pp. 78–85, 2014.
- [12] S. Malyshev, M. Krötzsch, L. González, J. Gonsior, and A. Bielefeldt, “Getting the most out of Wikidata: Semantic technology usage in Wikipedia’s knowledge graph,” in *The Semantic Web – ISWC 2018*, ser. LNCS, vol. 11137. Springer, 2018, pp. 376–394.
- [13] M. Grüninger and M. S. Fox, “Methodology for the design and evaluation of ontologies,” in *Proceedings of the IJCAI-95 Workshop on Basic Ontological Issues in Knowledge Sharing*, Montreal, Canada, 1995.
- [14] B. Motik, B. Cuenca Grau, I. Horrocks, Z. Wu, A. Fokoue, and C. Lutz, “OWL 2 web ontology language profiles (second edition),” W3C Recommendation, 2012. [Online]. Available: <https://www.w3.org/TR/owl2-profiles/>
- [15] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, “Retrieval-augmented generation for knowledge-intensive NLP tasks,” in *Advances in Neural Information Processing Systems 33 (NeurIPS 2020)*, 2020, pp. 9459–9474.
- [16] RDFLib Team, “RDFLib and OWL-RL: Python libraries for RDF and OWL 2 RL reasoning,” Software, versions 7.6.0 and 7.6.2, accessed October 2026. [Online]. Available: <https://github.com/RDFLib>
- [17] H. Paulheim, “Knowledge modeling,” Lecture slides, IE650 Knowledge Graphs, University of Mannheim, 2024.
- [18] A. Abid, A. Abdalla, A. Abid, D. Khan, A. Alfozan, and J. Zou, “Gradio: Hassle-free sharing and testing of ML models in the wild,” arXiv preprint arXiv:1906.02569, 2019. [Online]. Available: <https://arxiv.org/abs/1906.02569>
- [19] L. Feigenbaum, G. T. Williams, K. G. Clark, and E. Torres, “SPARQL 1.1 protocol,” W3C Recommendation, 2013. [Online]. Available: <https://www.w3.org/TR/sparql11-protocol/>